pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Analyzing Text, Part 2

WEEK 4, DAY 2, PART 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note

Part 1 proved the tools on one statement. Now my table: one row for each of the 46. How long each is, the rate range it states, how often a few plain words appear, and how much of its wording it shares with the statement before. Clean the text first so "Inflation" and "inflation." count as one word, and show me what each step does to the count. Counts only: no reading of policy.

The statements	
Source	Board of Governors of the Federal Reserve System, each statement from its press release: https://www.federalreserve.gov/monetarypolicy/fomccalendars.htm
Retrieved	2026-10-03
Span	46, 2021-01-27 to 2026-09-16, every scheduled meeting
Reuse	Public domain, citing the Board as the source: https://www.federalreserve.gov/disclaimer.htm
Changes	White space only

A quote prints the file's own words, with the statement's date.

Four columns: length, the range, word counts, wording shared.

17.7 Every statement's length, one column at a time

```
statements['text'].lower()
AttributeError: 'Series' object has no attribute 'lower'
```

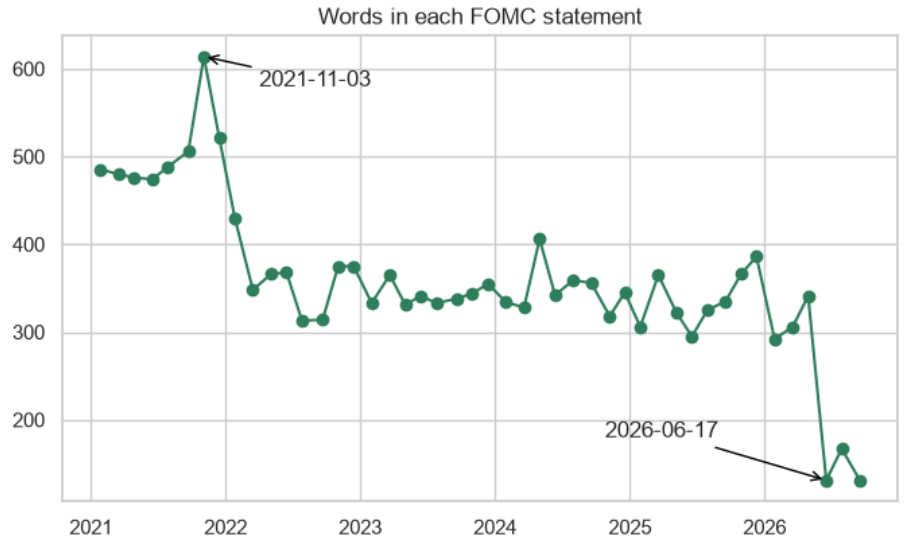
Read the last line, as on week 1's Day 3: a column is a Series, and a Series has no "lower()". The EDA notebook's ".str" runs a string method on every value.

```
lower_text = statements['text'].str.lower()
print(lower_text.iloc[0][:60])
the federal reserve is committed to using its full range of
```

```
statements['characters'] = statements['text'].str.len()
statements['words'] = statements['text'].str.split().str.len()
statements['meeting'] = pd.to_datetime(statements['date'], format='%Y-%m-%d')
statements['year'] = statements['meeting'].dt.year
print(round(statements['words'].mean(), 1), statements['words'].median())
359.3 344.5
```

359.3 against 344.5: a few long statements pull the mean above the median.

Length needed no cleaning. Counting a word does.



2021 ran longest; the last three, from 2026-06-17 on, run 131, 167 and 131 words. A length says how long, never why.

Excel side by side

With one statement per row, "=LOWER(C2)" and "=LEN(C2)" filled down do what ".str.lower()" and ".str.len()" do.

17.8 Cleaning text in steps

"Inflation" and "inflation" are one word to a reader and two strings to Python; so are "percent." and "percent".

```
print(sentence)
pieces = sentence.split()
print(len(pieces), pieces)
In support of these goals, the Committee decided to maintain the target range for the federal funds rate at 5-1/4 to 5-1/2 percent.
```

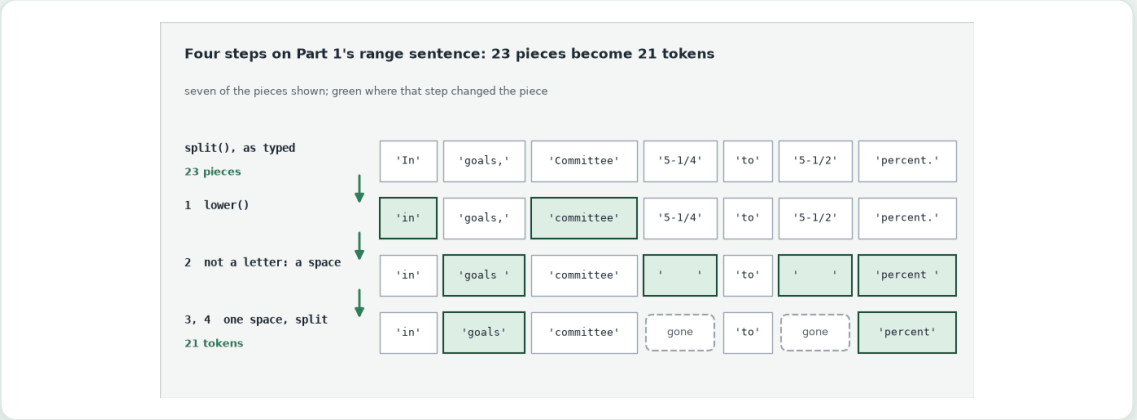
23 pieces: 'In' keeps its capital, 'goals,' its comma, 'percent.' its full stop.

```
step1 = sentence.lower()
step2 = re.sub(r'[^a-z\s]', ' ', step1)
print(step2)
in support of these goals  the committee decided to maintain the target range for the federal funds rate at       to       percent
```

Dropping the digits is a decision: Part 1 already took the figures from the text as published.

```
step3 = re.sub(r'\s+', ' ', step2).strip()
tokens = step3.split(' ')
print(len(tokens), tokens)
```

21 tokens: a token is one unit of text after cleaning, here a word. On a whole statement "\s" also takes the line breaks, so step 3 joins the paragraphs.



Four steps in one function, then the whole statement

```
def clean(text):
    text = re.sub(r'[^a-z\s]', ' ', text.lower())
    text = re.sub(r'\s+', ' ', text).strip()
    return text.split(' ')
print(clean(sentence) == tokens)
True
```

```
statement_tokens = clean(statement)
print(len(statement.split()), len(statement_tokens))
344 343
```

Excel side by side. A sentence in A2, each step in the next column, B to E

Step	Excel
Lower case	=LOWER(A2)
Letters and white space only	=REGEXREPLACE(B2, "[^a-z\s]", " "), Microsoft 365 only
One space between words	=TRIM(C2)
Tokens	=TEXTSPLIT(D2, " "), Microsoft 365 and Excel 2024

"TRIM" squeezes ordinary spaces, not a line break or "CHAR(160)"; without "REGEXREPLACE", step 2 is a nested "SUBSTITUTE" per character. On the whole statement, Part 1's "=COUNTA(TEXTSPLIT(A2, " "))" gives 340 words, not 344: the four line breaks between paragraphs are not spaces until "SUBSTITUTE(A2, CHAR(10), " ")" makes them so.

343 tokens, and more than a third of them say nothing.

17.9 Stopwords: the words that say nothing

```
from collections import Counter
counts = Counter(statement_tokens)
print(len(counts), counts.most_common(10))
180 [('the', 29), ('and', 16), ('to', 12), ('of', 11), ('committee', 11), ('inflation', 9), ('in', 7), ('policy', 7), ('monetary', 6), ('economic', 5)]
```

180 different tokens, and the top four, led by "the" at 29, are grammar. A "Counter" is a dictionary that counts: "counts['inflation']" is 9, and a token not there gives 0. Words that hold any sentence together and say nothing of its subject are stopwords.

"nltk", the Natural Language Toolkit, is in the course's "requirements.txt" and ships a longer list.

```
import nltk
nltk.download('stopwords', quiet=True)
from nltk.corpus import stopwords
stop_words = stopwords.words('english')
```

```
kept = [token for token in statement_tokens if token not in stop_words]
print(len(kept), len(set(statement_tokens)), len(set(kept)))
print(Counter(kept).most_common(5))
217 180 152
[('committee', 11), ('inflation', 9), ('policy', 7), ('monetary', 6), ('economic', 5)]
```

"committee" now leads with 11: a count, not a conclusion.

Excel side by side. The 343 tokens in B2:B344, nltk's list in column A of a sheet named "stop"

Job	Excel	Result
Count one token	=COUNTIF(B2:B344, "the")	29
Is it a stopword	=COUNTIF(stop!A:A, B2)>0, filled down	TRUE or FALSE
Keep the rest	=FILTER(B2:B344, COUNTIF(stop!A:A, B2:B344)=0)	217 tokens

"FILTER" is in Excel 2021 and Microsoft 365; "COUNTIF" in every version.

17.10 Stemming: one root for several forms

Stemming cuts off a word's ending, so "remain", "remained" and "remains" count as one; what is left is the stem. nltk's Porter stemmer applies fixed rules about English endings.

```
from nltk.stem import PorterStemmer
stemmer = PorterStemmer()
for word in ['remain', 'remained', 'remains', 'assess', 'assessing', 'assessments',
             'inflation', 'policy', 'committee', 'monetary', 'securities']:
    print(word, '->', stemmer.stem(word))
remain -> remain
remained -> remain
remains -> remain
assess -> assess
assessing -> assess
assessments -> assess
inflation -> inflat
policy -> polici
committee -> committe
monetary -> monetari
securities -> secur
```

The forms meet, but "polici" and "monetari" show that a stem is not a word: the rules look nothing up.

```
stems = [stemmer.stem(token) for token in kept]
print(len(stems), len(set(kept)), len(set(stems)))
print(Counter(stems).most_common(6))
217 152 146
```

Still 217, but 152 different fell to 146, and "remain" now counts 5.

Excel side by side

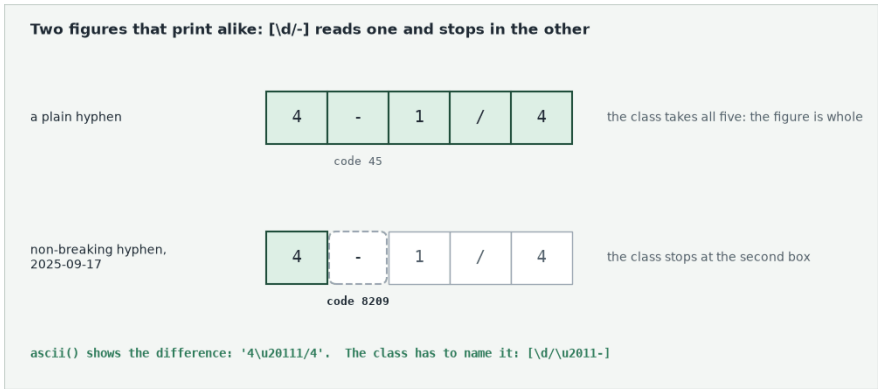
No function stems; "=COUNTIF(B2:B344, "remain*")" counts the tokens that start "remain": 5.

A stem count is rougher, so tokens it is. Now all 46, starting with the range.

17.11 The range in every statement

```
no_match = statements.loc[found.str.len() == 0, 'date']
print(no_match.tolist())
text = statements.loc[no_match.index[0], 'text']
piece = re.search(r'.{12} to \S+ percent', text).group(0)
print(piece)
print(ascii(piece))
e point to 4 to 4-1/4 percent
'e point to 4 to 4\u20111/4 percent'
```

Printed, the range looks like any other; "ascii()" shows "\u2011", the non-breaking hyphen, drawn like a hyphen so a line never breaks there. To Python it is another character, as hidden as the EDA notebook's trailing space.




```
wide_pattern = r'[\d/\u2011-]+ to [\d/\u2011-]+ percent'
found_wide = statements['text'].str.findall(wide_pattern)
print(found_wide.str.len().value_counts().sort_index())
statements['range_text'] = found_wide.str[0]
text
1    44
2     2
Name: count, dtype: int64
```

All 46 now give a range.

Excel side by side

Double-clicked, the CSV opens in the Windows code page and garbles the character; a copy written with "to_csv(..., encoding='utf-8-sig')" opens intact.

With that copy's statement of 2025-09-17 in C2

Job	Excel	Result
A character's code	=UNICODE(MID(C2, 715, 1))	8209;
		a plain hyphen gives 45
Swap it for a plain hyphen	=SUBSTITUTE(C2, UNICHAR(8209), "-")	4 to 4-1/4 percent

"UNICODE" and "UNICHAR" are in Excel 2013 and later.

17.12 Every statement cleaned and counted

```
def remove_stop_words(tokens):
    return [token for token in tokens if token not in stop_words]
statements['tokens'] = statements['text'].apply(clean)
statements['kept'] = statements['tokens'].apply(remove_stop_words)
```

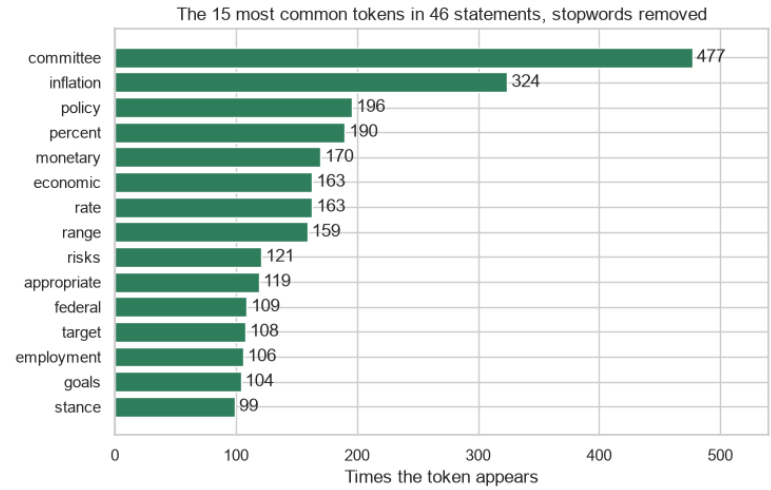
```
statements['n_tokens'] = statements['tokens'].str.len()
statements['n_kept'] = statements['kept'].str.len()
print(statements['n_tokens'].sum(), statements['n_kept'].sum())
statements.loc[statements['date'] == '2023-11-01', ['date', 'n_tokens', 'n_kept']]
16446 10204
```

16,446 tokens, 10,204 kept, and 2023-11-01 ties with the hand counts of 17.8 and 17.9.

```
print(lower_text.str.contains(r'\bbalance\b').sum(), lower_text.str.contains('balance').sum())
statements['inflation'] = lower_text.str.count(r'\binflation\b')
print(statements['inflation'].iloc[0], statements['inflation'].min(),
      statements['inflation'].max(), statements['inflation'].sum())
25 29
10 1 10 324
```

25 hold the word, 29 the letters, which "imbalances" also holds.

10,204 tokens, and the row for 2023-11-01 ties. Next, the counts as a table.



507 different tokens, and "inflation"'s 324 matches the column above.

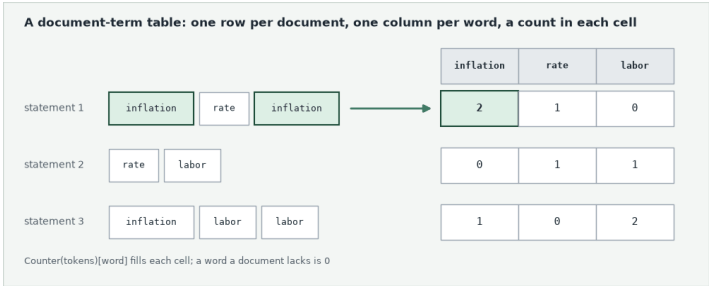
Excel side by side

".apply()" is a formula filled down a column. With the texts in C2:C47, "=SUMPRODUCT(--ISNUMBER(SEARCH("balance", C2:C47)))" gives 29, the letters; "=(LEN(C2)-LEN(SUBSTITUTE(LOWER(C2), "inflation", "")))/LEN("inflation")" gives 10 for 2021-01-27.

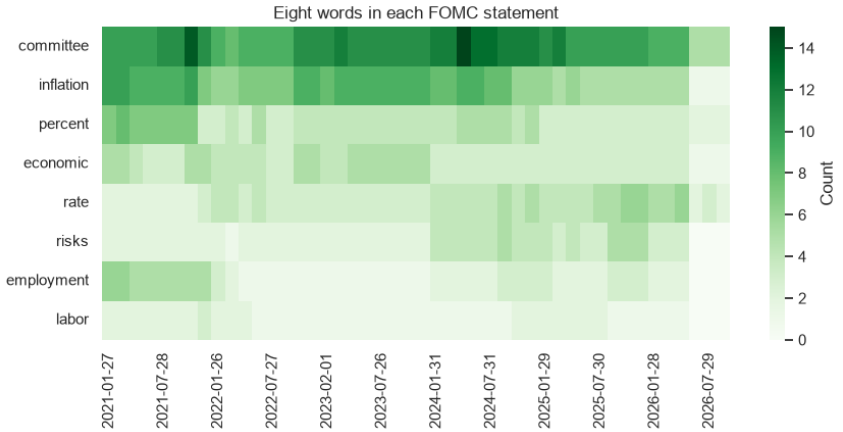
17.13 A document-term table

A document-term table has a row per document, a column per word, and a count in each cell: the way text reaches anything that works on numbers.

```
terms = ['committee', 'inflation', 'percent', 'economic',
         'rate', 'risks', 'employment', 'labor']
counters = statements['kept'].apply(Counter)
term_table = pd.DataFrame(index=statements['date'])
for term in terms:
    term_table[term] = [counter[term] for counter in counters]
print(term_table.iloc[0].tolist())
print(term_table.sum().tolist())
[10, 10, 7, 5, 2, 2, 6, 2]
[477, 324, 190, 163, 163, 121, 106, 62]
```



Across the 46, the whole word rate appears 163 times: keep that number.



The last three columns are paler in every row: the three shortest statements, so a count leans on length.

Excel side by side

Conditional Formatting, Color Scales on the table of counts draws the same picture.

Per 100 tokens by year, and the share that hold a word

```
per_100 = term_table.div(statements['n_kept'].to_numpy(), axis=0) * 100
per_100_by_year = per_100.groupby(statements['year'].to_numpy()).mean()
per_100_by_year.style.format('{:.2f}').background_gradient(cmap='Greens', axis=0)
```

"inflation" runs 2.93 per 100 in 2021, 4.13 in 2023, 1.91 in 2026: a count over the total, as a market value weight is.

	committee	inflation	percent	economic	rate	risks	employment	labor
2021	3.47	2.93	2.13	1.32	0.68	0.64	1.69	0.68
2022	4.17	3.23	1.61	1.76	1.50	0.83	0.60	0.61
2023	5.18	4.13	1.86	2.21	1.40	0.93	0.47	0.47
2024	5.94	3.64	2.17	1.42	2.01	1.95	1.13	0.60
2025	5.05	2.57	1.46	1.46	2.30	1.94	1.22	0.80
2026	5.31	1.91	1.97	1.38	2.74	0.80	0.53	0.27

```
holds = pd.Series(['balance' in tokens for tokens in statements['kept']])
(holds.groupby(statements['year'].to_numpy()).mean() * 100).round(1)
2021      0.0
2022     75.0
2023      0.0
2024    100.0
2025    100.0
2026     50.0
dtype: float64
```

None in 2021 or 2023, all of 2024 and 2025: a share counts a statement once, however often it says the word.

46 by 507; the top 50 cover 48.0 percent, and 70 tokens appear once. scikit-learn's "CountVectorizer" builds this in one step, "max_features" keeping the most common.

Excel side by side

A pivot: statement in Rows, token in Columns, count in Values. With the year in D2:D47 and TRUE or FALSE in E2:E47, "=COUNTIFS(D2:D47, 2026, E2:E47, TRUE)/COUNTIF(D2:D47, 2026)" gives 0.5.

17.14 How much wording each statement keeps

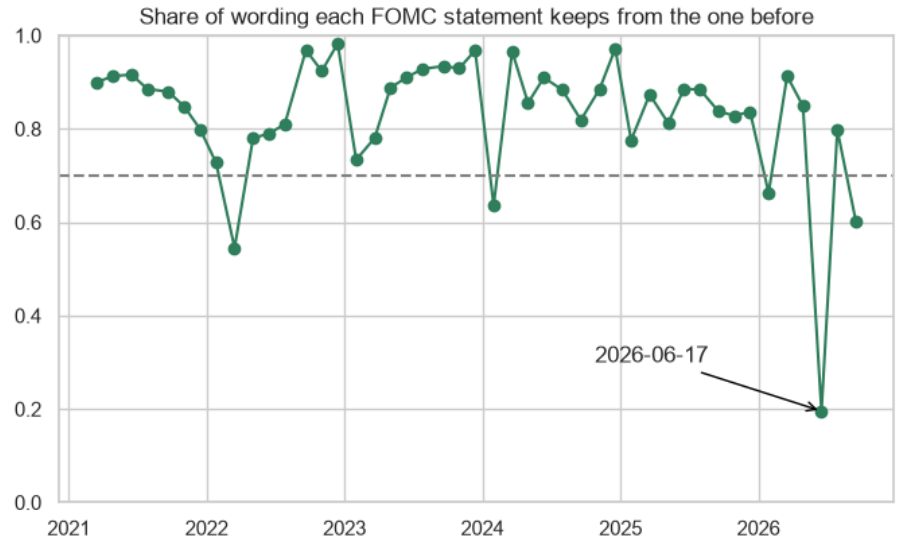
The Jaccard similarity of two statements is the different tokens in both over the different tokens in either: 1 for the same words, 0 for none shared.

```
values = [None]
for i in range(1, len(statements)):
    values.append(similarity(statements['kept'].iloc[i], statements['kept'].iloc[i - 1]))
statements['similarity'] = values
low = statements.loc[statements['similarity'].idxmin()]
high = statements.loc[statements['similarity'].idxmax()]
print(round(statements['similarity'].mean(), 3), low['date'], round(low['similarity'], 3),
      high['date'], round(high['similarity'], 3))
print(statements.loc[statements['similarity'] < 0.7, 'date'].tolist())
0.832 2026-06-17 0.196 2022-12-14 0.982
['2022-03-16', '2024-01-31', '2026-01-28', '2026-06-17', '2026-09-16']
```

Three of the five below 0.7 fall in 2026.

Excel side by side

"=UNIQUE(A2:A6)" lists each word once (Excel 2021 and Microsoft 365).



The first statement has no point: nothing comes before it. A low point says many words changed, not which, and not why.

Predict first: decide, then run each in the notebook

Predict 1

The statement holds the phrase "the Committee's goals." How many tokens does "clean()" return for it? A) 3: "the", "committees", "goals". B) 3: "the", "committee", "goals". C) 4.

```
phrase = re.search(r"the \w+'s \w+\.", statement).group(0)
print(phrase, clean(phrase))
```

Predict 2

Does the Porter stemmer give "economic" and "economy" the same stem? A) Yes: both become "econom". B) No: two different stems. C) Neither changes: both stay as typed.

```
for word in ['economic', 'economy', 'production', 'productivity', 'time', 'timing']:
    print(word, '->', stemmer.stem(word))
```

Predict 3

Part 1's "range_pattern" found the range in 2023-11-01. On all 46 statements, how many give no match at all? Write down a number.

```
found = statements['text'].str.findall(range_pattern)
print(found.str.len().value_counts().sort_index())
```

Predict 4

"a = ['inflation', 'remains', 'elevated']" and "b = ['inflation', 'remains', 'somewhat', 'elevated', 'inflation']". What is their similarity? Write down a number.

```
def similarity(first, second):
    both = set(first) & set(second)
    either = set(first) | set(second)
    return len(both) / len(either)
a = ['inflation', 'remains', 'elevated']
b = ['inflation', 'remains', 'somewhat', 'elevated', 'inflation']
print(similarity(a, b))
```

The note, answered

The note asked	Answer
How long each is	131 to 614 words
The range it states	46 of 46
A few plain words	10,204 kept tokens; 2023-11-01 ties at 343 and 217
Wording shared	a mean of 0.832, a low of 0.196

date	words	rate_range	n_kept	inflation	similarity
2026-01-28	292	3-1/2 to 3-3/4 percent	178	5	0.663
2026-03-18	306	3-1/2 to 3-3/4 percent	186	5	0.913
2026-04-29	341	3-1/2 to 3-3/4 percent	202	5	0.849
2026-06-17	131	3-1/2 to 3-3/4 percent	80	1	0.196
2026-07-29	167	3-1/2 to 3-3/4 percent	101	1	0.798
2026-09-16	131	3-3/4 to 4 percent	79	1	0.602

Next, the case study asks whether a statement's wording can become one number, and how far to trust it.